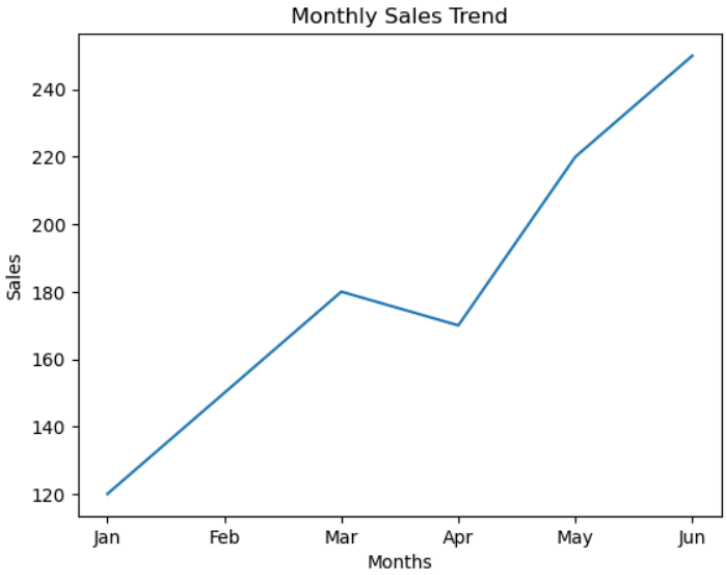
EXPERIMENT 5
DATA VISUALIZATION USING MATPLOTLIB

Aim
To create and analyze various data visualizations such as Line Charts, Bar Charts, Pie Charts, Histograms, Scatter Plots, and Box Plots using Matplotlib for effective data interpretation.

Algorithm 1 -Line Chart
Step 1 Import Matplotlib
Step 2 Create dataset
Step 3 Use plot() function
Step 4 Add title and labels
Step 5 Display chart

Program 1:
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [120, 150, 180, 170, 220, 250]
plt.plot(months, sales)
plt.title("Monthly Sales Trend")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```

Output
Monthly Sales Trend
Sales growth over six months.



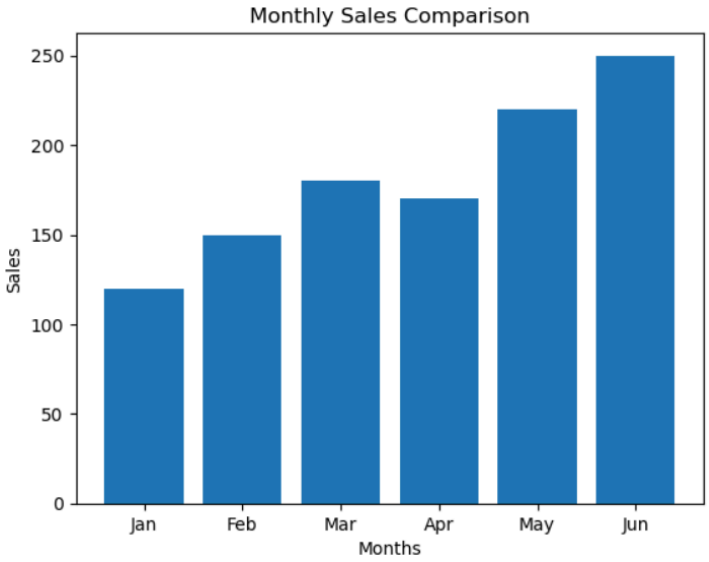
Interpretation
Sales gradually increased from January to June.
Highest Sales = June (250) Lowest Sales = January (120)

Algorithm 2 -Bar Chart
Step 1 Create dataset
Step 2 Use bar() function
Step 3 Add chart labels
Step 4 Display chart

Program 2:
```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [120, 150, 180, 170, 220, 250]
plt.bar(months, sales)
plt.title("Monthly Sales Comparison")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```

Output:
Title: Monthly Sales Comparison

A bar chart comparing sales across different months.



Interpretation
The bar chart provides a clear comparison of monthly sales. January has the lowest sales of 120, while June has the highest sales of 250. Sales increased overall from January to June.

Algorithm 3: Pie Chart

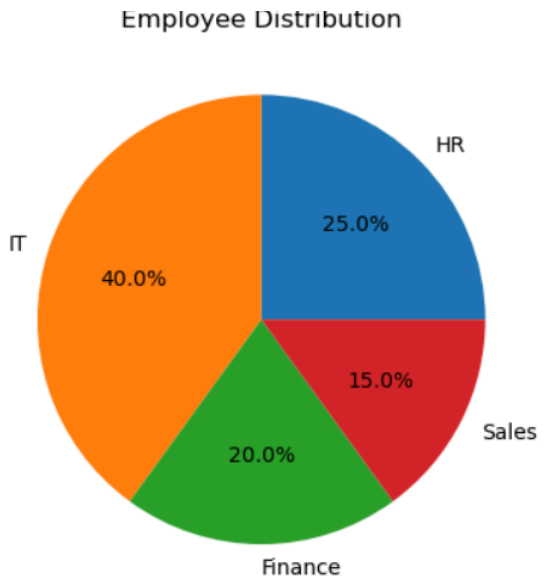
Step 1: Create the department and employee data.
Step 2: Use the pie() function to create a pie chart.
Step 3: Add department names as labels.
Step 4: Display percentages using the autopct parameter.
Step 5: Add a title and display the chart.

Program 3:

```
departments = ["HR", "IT", "Finance", "Sales"]

employees = [25, 40, 20, 15]

plt.pie(employees,labels=departments,autopct="%1.1f%%")

plt.title("Employee Distribution")

plt.show()
```

Output:
Title: Employee Distribution

A pie chart showing the percentage distribution of employees across HR, IT, Finance, and Sales departments.



Interpretation:
The IT department has the highest number of employees and represents 40% of the total employees. HR represents 25%, Finance represents 20%, and Sales represents 15%. The chart clearly shows the proportion of employees in each department.

Algorithm 4: Histogram

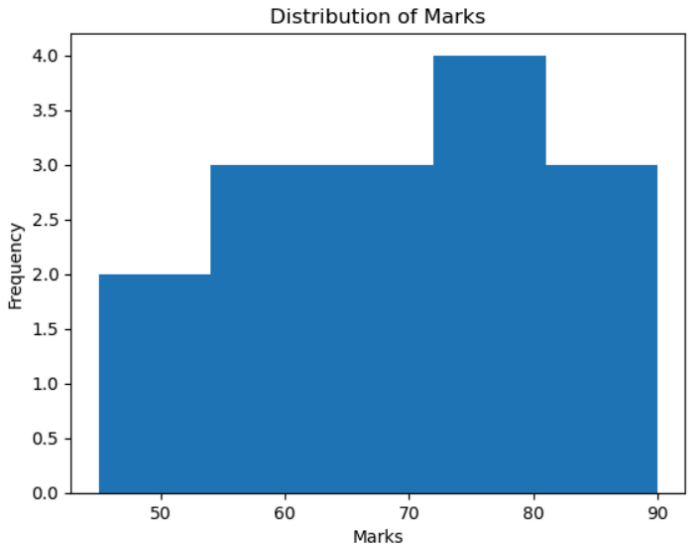
Step 1: Create a numerical dataset containing student marks.
Step 2: Use the hist() function to create a histogram.
Step 3: Divide the data into suitable intervals using bins.
Step 4: Add a title and label the X-axis and Y-axis.
Step 5: Display the histogram using show().

Program 4:

```
marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 90]

plt.hist(marks, bins=5)

plt.title("Distribution of Marks")

plt.xlabel("Marks")

plt.ylabel("Frequency")

plt.show()
```

Output:
Title: Distribution of Marks

A histogram showing the frequency distribution of students' marks.



Interpretation:
The histogram shows how the marks are distributed across different intervals. Most marks fall

within the middle-to-higher range. The histogram helps understand the distribution and frequency of numerical data

Algorithm 5: Scatter Plot

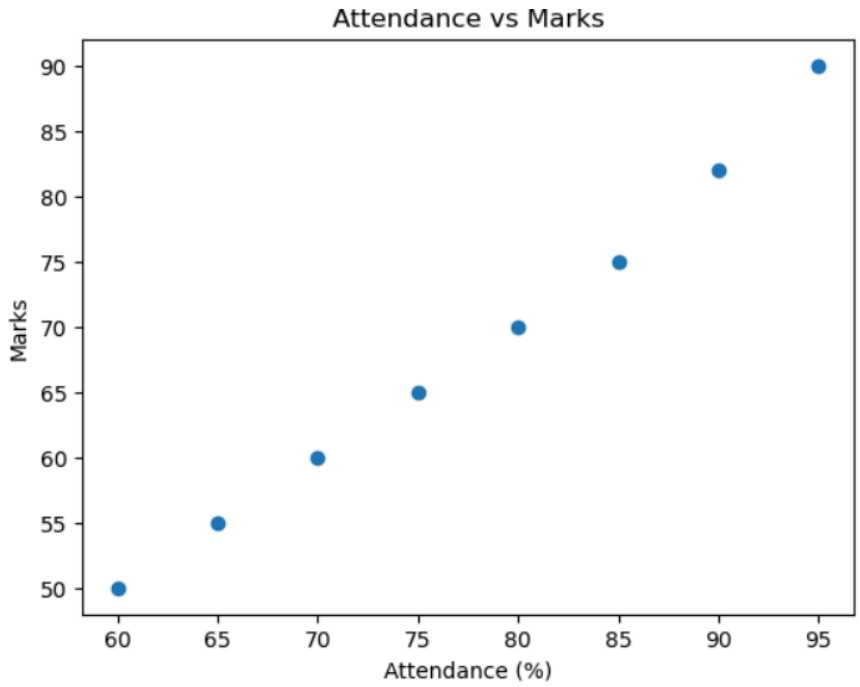
Step 1: Create two numerical datasets for attendance and marks.
Step 2: Use the scatter() function to create a scatter plot.
Step 3: Plot attendance on the X-axis and marks on the Y-axis.
Step 4: Add a suitable title and axis labels.
Step 5: Display the scatter plot using show().

Program 5:

```
attendance = [60, 65, 70, 75, 80, 85, 90, 95]

marks = [50, 55, 60, 65, 70, 75, 82, 90]

plt.scatter(attendance, marks)

plt.title("Attendance vs Marks")

plt.xlabel("Attendance (%)")

plt.ylabel("Marks")

plt.show()
```

Output:
Title: Attendance vs Marks

A scatter plot showing the relationship between student attendance and marks.



Interpretation:
The points show a positive relationship between attendance and marks. Students with higher attendance generally have higher marks. The scatter plot helps identify the relationship between two numerical variables.

Algorithm 6: Box Plot

Step 1: Create a numerical dataset containing salary values.
Step 2: Use the boxplot() function to create a box plot.
Step 3: Add a suitable title and label the Y-axis.
Step 4: Display the box plot using show().
Step 5: Identify values that appear outside the normal range.

Program 6:

```
salary = [25000, 27000, 30000,32000, 35000, 40000,150000]

plt.boxplot(salary)

plt.title("Salary Distribution")

plt.ylabel("Salary")

plt.show()
```

Output:
Title: Salary Distribution

A box plot showing the distribution of salaries and identifying unusual values.

Box Plot Components:Minimum, First Quartile (Q1), Median, Third Quartile (Q3), Maximum, Outlier

Interpretation:
Most salary values lie within the normal range. The salary of 150000 appears as an outlier. The box plot helps identify the spread, distribution, and unusual values in the salary dataset.

Algorithm 7: Multiple Line Charts

Step 1: Create datasets for monthly sales and profit.
Step 2: Use the plot() function to create the sales line.
Step 3: Use the plot() function again to create the profit line.
Step 4: Add labels and a legend to distinguish the two lines.
Step 5: Add a title and display the chart using show().

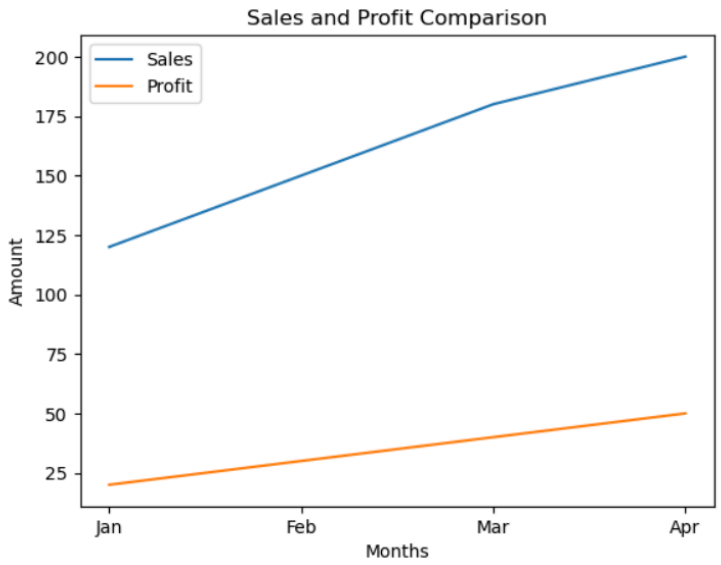
Program 7:

```
months = ["Jan", "Feb", "Mar", "Apr"]

sales = [120, 150, 180, 200]

profit = [20, 30, 40, 50]

plt.plot(months, sales, label="Sales")

plt.plot(months, profit, label="Profit")

plt.title("Sales and Profit Comparison")
```

```
plt.xlabel("Months")

plt.ylabel("Amount")

plt.legend()

plt.show()
```

Output:
Title: Sales and Profit Comparison

A multiple line chart showing the trends of Sales and Profit from January to April.



Interpretation:
Both sales and profit show an increasing trend. Sales increased from 120 in January to 200 in April, while profit increased from 20 to 50. The chart allows easy comparison of two datasets over the same period.

Result

Thus, various visualization techniques such as Line Charts, Bar Charts, Pie Charts, Histograms, Scatter Plots, and Box Plots were successfully created using Matplotlib. The graphical representations helped in understanding trends, distributions, comparisons, and relationships within the dataset.